ChemE Jeopardy Web App

Arturo Arias | Last updated: 2026-05-17

Overview

This is a browser-based game platform designed to support live ChemE Jeopardy games for chemical engineering review sessions, academic competitions, and classroom practice. It replaces a single-computer presentation workflow with a shared web application that can be run locally on a classroom network or temporarily hosted online for a live event. The game flow and default rule assumptions are based on AIChE ChemE Jeopardy competition materials [1].

The system starts from an admin-managed home page where the event host creates game rooms only when needed. Each room has its own moderator console, player console, and read-only display screen, so multiple rooms can run in parallel from the same server instance.

For official rules and further information about how ChemE Jeopardy is played, see the AIChE ChemE Jeopardy competition page.

Who This Helps

Chemical Engineering Students

Students can participate from phones or laptops, join a team, buzz in, and submit wagers or Final Jeopardy responses without needing special software.

Professors And Teaching Teams

Instructors can run review games using prepared question files, edit the setup before class, and make live scoring decisions from one moderator console.

Core Screens

Screen	Purpose	Typical URL
Admin	Creates and manages game rooms, including room title, URL slug, passwords, and initial question source.	`/` or `/games`
Moderator	Login-protected setup, team rosters, clue selection, buzzing control, judging, scoring, question upload, and game flow.	`/game-1/moderator`
Player	Team join, clock synchronization, focused buzz submission, optional board/scores panels, Daily Double wagers, and Final Jeopardy input.	`/game-1/player`
Display	Read-only room screen that shows a team lobby before start, then board, scores, clue prompts, timers, and answer reveal.	`/game-1/display`

Tools And Technologies

- Java 21 with the JDK embedded HTTP server for the backend.
- Plain HTML, CSS, and JavaScript for the browser interfaces.
- Server-Sent Events for live state updates to browser screens.
- JSON files and optional `.chemej` image packages for game definitions and question sources.
- Java executable jar for local question authoring, CSV conversion, and image packaging.
- PowerShell helper scripts for optional local Windows build/run workflows.
- Dockerfile for portable cloud deployment when needed.

Design And Architecture

The application is intentionally dependency-light. The server serves static pages and exposes JSON endpoints. The game manager owns room creation and per-room authentication, while each game engine owns its own rules and runtime state.

Component	Responsibility
`GameEngine`	Encapsulates game rules, phases, scoring, timers, Final Jeopardy, tie-breakers, and source-file persistence.
`AppServer`	Registers routes, serves static files, parses forms, writes JSON, and broadcasts live events.
`AuthManager`	Controls moderator sessions and player join password checks.
`AppConfig`	Loads deployment settings from command-line arguments, environment variables, and optional local `.env`.
`Json`	Parses and serializes the small JSON shapes used by the game without external libraries.

Question Source Workflow

A game room can start from the default JSON source file, `data/game-definition.json`, or from a runtime upload chosen when the room is created. The file includes teams, board categories, clue prompts, official responses, point values, Daily Double flags, Final Jeopardy content, tie-breakers, timers, and optional clue image metadata.

Uploaded question files are runtime-only and scoped to one game room. The app accepts normal JSON files, plus `.chemej` or ZIP image packages containing `game.json` and an `images/` folder. Only one runtime upload is kept per room; uploading another one replaces the previous file.

The repository also includes a local Java authoring jar under `tools/`. The primary entry point is a Swing GUI that lets the host import an existing JSON into a manual question grid, create a new grid, choose CSV or JSON inputs with file pickers, choose an output folder, and package local clue images into the app's compressed upload format. Command-line modes inside the same jar remain available for repeatable CSV conversion and package creation.

The editable Java source for the authoring tool can be kept locally under `tools-src/`, which is ignored by Git. The GitHub-facing artifact is the compiled jar in `tools/`.

Development Process

- Defined the ChemE Jeopardy rule flow and mapped it to a small state machine.
- Built a Java server that serves all pages and APIs from one process.
- Created separate browser experiences for moderator, players, and room display.
- Added an admin game manager so no room exists until the host creates one.
- Added independent game-room URLs and passwords for parallel rooms.
- Added player clock synchronization to reduce buzz-order bias from network latency.
- Added moderator and player access control for online use.

- Added runtime question uploads, image packages, and a local Java GUI authoring jar.
- Added display lobby rosters, focused player buzzing, and configurable color themes.
- Prepared the project for GitHub release with Docker, README, license, and generated-file ignores.

Deployment Readiness

The app can run locally with the JDK or as a Docker container. Docker is useful for platforms such as Google Cloud Run, Azure Container Apps, Railway, Render, or Fly.io. A custom domain is not required because cloud platforms provide default HTTPS URLs.

Because live game state is kept in memory, production-style hosting should use one instance or replica. For a one-day event, the app can be started before the session, used during the event, and turned off afterward.

Scope and Security

The application is intended for controlled classroom, review-session, or temporary event environments rather than open-ended public internet hosting. Access is protected with moderator and player passwords, and secrets are loaded from environment configuration rather than committed files.

For live events, hosts should use fresh passwords, share links only with participants, and shut down hosted instances after use. Public repository question files should be treated as samples because they may include answer keys. Permanent public deployments should add additional protections such as rate limiting, stricter upload/body limits, HTTPS-only secure cookies, and restricted server-side file loading.

References

[1] AIChE, "ChemE Jeopardy Competition," American Institute of Chemical Engineers. Accessed: May 4, 2026. [Online]. Available: https://www.aiche.org/community/awards/cheme-jeopardy-competition